Comprehensive Test Plan - Thermal Simulation Program

fs881 bta26

Legend for IDs

U = Unit test (single class or method)
I = Integration test (interfaces between classes)
S = System test (end-to-end behaviour)
E = Exceptional / invalid inputs (robustness)

Unit Tests

ID	Purpose	Steps	I/O or Conditions	Expected Result	Pass/Fail Criteria
U1	Create Component correctly	1) Create Component("MOSFET", 4.0, 100)	name="MOSFET", Rth=4.0, Tmax=100	Attributes stored exactly	Pass if attributes match inputs
U2	Component.is_safe nominal safe	1) c=Component(..., Tmax=100) 2) call c.is_safe(99.9)	temperature=99.9	returns True	Pass if True
U3	Component.is_safe boundary	1) c.is_safe(100)	temperature == Tmax	returns True	Pass if True
U4	Component.is_safe unsafe	1) c.is_safe(100.01)	temperature > Tmax	returns False	Pass if False
U5	DutyCycle initialises cycle_time	1) d=DutyCycle(5,5,12)	on=5, off=5	cycle_time=10	Pass if cycle_time correct
U6	DutyCycle.power_at_time ON region start	1) d.power_at_time(0)	t=0, on=5/off=5	returns power_on	Pass if 12
U7	DutyCycle.power_at_time ON region inside	1) d.power_at_time(4.999)	t < on_time	returns power_on	Pass if 12
U8	DutyCycle.power_at_time boundary switch	1) d.power_at_time(5)	t == on_time	returns 0	Pass if 0
U9	DutyCycle.power_at_time OFF region	1) d.power_at_time(7)	5 <= t < 10	returns 0	Pass if 0
U10	DutyCycle repeats correctly (mod behaviour)	1) d.power_at_time(12)	cycle_time=10, t%10=2	returns power_on	Pass if 12
U11	ThermalModel stores ambient+component	1) create Component 2) m=ThermalModel(25,c)	ambient=25	attributes stored	Pass if stored correctly
U12	ThermalModel.next_temperature steady-state target	1) target=Ta + P*Rth	Ta=25, P=10, Rth=4	target=65	Pass if computed target is 65
U13	ThermalModel.next_temperature increases when below target	1) current=25 2) call next_temperature(current, P>0, dt=1)	current < target	new_temp > current	Pass if temp rises
U14	ThermalModel.next_temperature decreases when above target	1) current=80 2) call with P=0, dt=1	target=ambient=25	new_temp < current	Pass if temp falls

ID	Purpose	Steps	I/O or Conditions	Expected Result	Pass/Fail Criteria
U15	ThermalModel dt effect	1) compute next_temp with dt=0.5 vs dt=2.0	same current/P	dt=2 changes more than dt=0.5	Pass if larger dt causes larger change
U16	ResultsAnalyser max_temperatures basic	1) Provide small fake history	history=[(0,[25,25]),(1,[30,28])]	returns [30,28]	Pass if matches
U17	ResultsAnalyser max_temperatures with constant	1) history temps constant all 25	all 25	returns [25,...]	Pass if equals constants
U18	ResultsAnalyser max_temperatures empty history	1) history=[]	empty	Returns [0, 0, ...]	Pass if list of zeros returned
U19	ResultsAnalyser thermal_margins nominal	1) components Tmax [100,90] 2) max_temps [30,50]	margin = Tmax - max	returns [70,40]	Pass if correct
U20	ResultsAnalyser thermal_margins negative margin	1) max_temp > Tmax	unsafe	margin negative	Pass if negative values appear correctly
U21	ScenarioManager.plot handles 1 scenario	1) add one scenario 2) plot	one dataset	plot appears with one line and label	Pass if plot renders without error
U22	ScenarioManager.plot handles multiple scenarios	1) add 2+ scenarios	multiple datasets	plot shows multiple labelled lines	Pass if plot renders and legend contains both names

Integration Tests

ID	Purpose	Steps	I/O or Conditions	Expected Result	Pass/Fail Criteria
I1	Simulation initialises temperature list correctly	1) Create 2 components + 2 models 2) Run sim for 0 seconds	duration=0	history has at least one entry at t=0, temps list length=2	Pass if history contains (0, [T1,T2]) and len temps matches
I2	Simulation updates all components each step	1) Create 2 models with different Rth 2) Run multiple steps	same duty cycle P	component temps diverge due to different Rth	Pass if temps differ after some steps
I3	Simulation stores history as tuples	1) Run sim 2) check history format	history entries	each entry is (t, [temps])	Pass if structure matches
I4	Simulation uses copy of temperatures	1) Run sim 2) check earlier history temps do not change later	copy behaviour	earlier entries remain unchanged	Pass if earlier temps stable
I5	Simulation stop_reason Completed	1) Choose low power and high Tmax 2) run full duration	safe all run	stop_reason="Completed"	Pass if Completed and full duration reached
I6	Simulation stops on overtemperature	1) Choose high power, low Tmax 2) run	unsafe occurs	returns early, stop_reason starts with "Overtemperature:"	Pass if stop is early and reason correct
I7	Safety check identifies which component failed first	1) Make component A low Tmax 2) component B high Tmax	A should fail first	stop_reason contains A name	Pass if correct component name appears

ID	Purpose	Steps	I/O or Conditions	Expected Result	Pass/Fail Criteria
I8	ScenarioManager stores results structure	1) run_scenario("A", sim, comps)	normal run	results_db["A"] has keys: history, max_temperatures, thermal_margins, stop_reason	Pass if keys exist and types correct
I9	ScenarioManager overwriting scenario name	1) run_scenario("A", sim1) then run_scenario("A", sim2)	same key	latest results overwrite	Pass if results_db["A"] equals second run
I10	ScenarioManager multiple scenarios	1) run "A" and "B"	2 keys	results_db has both	Pass if both present and distinct

System Tests

ID	Purpose	Steps	I/O or Conditions	Expected Result	Pass/Fail Criteria
S1	End-to-end: single component, 50% duty	1) Create 1 component 2) duty 5/5, P=12 3) run scenario manager	nominal	results_db filled, stop_reason Completed	Pass if no crash and outputs plausible
S2	End-to-end: compare 50% vs 80% duty	1) scenario A 5/5 2) scenario B 8/2 3) plot	two scenarios	higher duty shows higher temps	Pass if B curve above A most of time
S3	End-to-end: overtemperature scenario	1) lower Tmax to 40 2) run	unsafe	stop_reason indicates overtemp, history ends early	Pass if early stop and correct reason
S4	End-to-end: multi-component list behaviour	1) 3 components 3 models list 2) run	list handling	history stores temps list length=3; max_temps length=3; margins length=3	Pass if all lengths match number of components
S5	End-to-end: results consistency	1) run scenario 2) compute max_temps 3) verify each max temp	consistency	values align	Pass if computed max equals direct max from history
S6	End-to-end: plot uses first component only	1) multi-component sim 2) plot	plot logic	only temps[0] is plotted	Pass if plot matches component 0 behaviour

Exceptional / Invalid Input Tests

ID	Purpose	Steps	I/O or Conditions	Expected Result	Pass/Fail Criteria
E1	DutyCycle with on_time=0	1) DutyCycle(0,5,12) 2) power_at_time(0..10)	on=0	always OFF	Pass if always returns 0
E2	DutyCycle with off_time=0	1) DutyCycle(5,0,12) 2) power_at_time(0..10)	off=0	always ON	Pass if always returns 12
E3	DutyCycle invalid negative times	1) DutyCycle(-1,5,12)	on_time negative	No exception raised. Behaviour is non-physical.	Pass if no crash
E4	DutyCycle cycle_time=0 case	1) DutyCycle(0,0,12) 2) call power_at_time(1)	cycle_time=0	Raises ZeroDivisionError	Pass if ZeroDivisionError occurs
E5	Simulation with time_step=0	1) run Simulation(..., time_step=0)	dt=0	Infinite loop or ValueError	Pass if error raised and no infinite loop

ID	Purpose	Steps	I/O or Conditions	Expected Result	Pass/Fail Criteria
E6	Simulation with negative duration	1) duration=-1	invalid	Returns empty history [] and stop_reason Completed	Pass if history empty and reason is Completed
E7	ThermalModel with negative thermal_resistance	1) Rth=-2	physically invalid	No exception raised. Temperature trend non-physical.	Pass if code runs without crashing
E8	ScenarioManager.plot with empty results_db	1) create manager 2) call plot()	no scenarios	Displays empty plot without crashing	Pass if no exception occurs